

Neural Network Project Design: Multi-Modal Medical Diagnostics

1. Project Summary & End Goals

This document outlines the design for a Neural Networks course group project focused on developing an advanced diagnostic system for healthcare. The project shifts away from single-modality image classification by implementing a **Multi-Modal Fusion Architecture**. By integrating high-dimensional visual data (Chest X-Rays) with structured clinical data (patient metadata, lab results, and vitals), the system mirrors real-world clinical decision-making.

End Goals:

- **Primary Objective:** Accurately classify the presence of pulmonary diseases or predict patient readmission risks by simultaneously processing scans and clinical history.
- **Technical Objective:** Design, train, and validate a custom Fusion Layer that effectively bridges Convolutional Neural Networks (CNN) and Multi-Layer Perceptrons (MLP).
- **Analytical Objective:** Conduct a rigorous ablation study demonstrating the performance delta between a multi-modal approach versus isolated single-modality baselines (Image-only vs. Clinical-only).

2. System Design and Architecture

The system utilizes a two-branch architecture converging into a unified classification head.

2.1. Multi-Modal Fusion Architecture

Component	Input Data Type	Network Type	Function
Vision Branch	Image (Chest X-Ray)	Convolutional Neural Network (CNN)	Extracts high-level spatial anomalies from radiological scans (e.g., opacities, structural shifts).

Clinical Branch	Tabular (Float/Integer/Categorical)	Multi-Layer Perceptron (MLP)	Processes structured patient metadata, lab results, and demographic health markers.
Fusion Layer	Concatenated Feature Vectors	Dense/Fully Connected Layers	Learns the cross-modal relationships (e.g., correlating a faint visual opacity with an elevated white blood cell count).

2.2. Data Pipeline

- **Image Preprocessing:** Chest X-rays will be resized, normalized, and augmented (rotations, slight crops) to prevent overfitting on the vision branch.
- **Tabular Preprocessing:** Clinical metadata will undergo missing-value imputation, categorical encoding (e.g., one-hot encoding for admission types), and standard scaling for continuous physiological variables.
- **Custom DataLoader:** A custom PyTorch `Dataset` class will be implemented to yield paired (`image_tensor`, `clinical_vector`) tuples for each unique patient ID during the training loop.

3. Tools and Technologies

3.1. Core Development Stack

- **Programming Language:** Python 3
- **Deep Learning Framework:** PyTorch (selected for research flexibility and custom layer development).
- **Data Manipulation & Preprocessing:** Pandas for clinical data wrangling; Scikit-learn for imputation and feature scaling.

3.2. Model Architectures

- **Vision Branch:** Pre-trained architectures such as ResNet-50 or EfficientNet (via transfer learning) fine-tuned on the CXR data.
- **Clinical Branch:** Standard Multi-Layer Perceptron (MLP) with dropout layers for regularization.
- **Fusion Layer:** Custom-built concatenation layer followed by fully connected layers utilizing ReLU activation.

3.3. Development Resources

- **Environment:** Windows Subsystem for Linux (WSL) for a native Linux development workflow.
- **Version Control:** Git/GitHub for collaborative code management and version tracking across the team.
- **Dataset:** "Multi-Modal Healthcare Dataset: Patient Records" (hosted on Kaggle), specifically utilizing the Chest X-Ray directory and the corresponding Patient Metadata CSV.
- **Documentation:** LaTeX for compiling the final academic report and formatting complex mathematical/architectural diagrams.

4. Team Logistics and Task Delegation

With a five-person team, the development pipeline will be parallelized to meet course deadlines.

Task Description	Core Focus
1. Data Engineering & Pipeline	Developing the PyTorch DataLoader, cleaning the Pandas dataframe, and aligning the CXR images with patient IDs.
2. Vision Branch (CNN)	Setting up transfer learning, image augmentation, and establishing the vision-only baseline. Deconstruct the model by removing the final classification head to repurpose the architecture as a Feature Extractor that outputs a high-dimensional visual context vector.
3. Clinical Branch (MLP)	Architecting the MLP, feature selection, and establishing the tabular-only baseline. Build a Multi-Layer Perceptron (MLP) to process tabular data, including Age, Gender, Blood Type, Medical Condition, Admission Type, and Medication .

	Apply One-Hot Encoding to categorical variables and Standardization to numerical variables to generate the clinical context vector.
4. Fusion Architecture & Tuning	Designing the concatenation logic, hyperparameter tuning, and running the ablation study. Develop a Fusion Layer to perform feature concatenation of visual context vector and clinical context vector Design a Regression Head (dense layers) to map the fused vector to a continuous scalar value representing the predicted number of days for the patient's stay. Freeze the weights of the Vision Branch to protect the pre-trained spatial knowledge from initial gradient instability, then exclusively train the Clinical Branch, Fusion Layer, and Regression Head to align the clinical features with the target regression objective. Unfreeze the Vision Branch and initiate joint training across the entire architecture. Predict Length of Stay (Discharge Date - Date of Admission)
5. Technical Writing & Validation	Synthesizing results, generating confusion matrices/F1-scores, and formatting the final LaTeX document and presentation slides.